

Tema 4: NP-completitud (1ª Parte)

Serafín Moral

Universidad de Granada

Abril, 2025

- Reducción
- Completitud
- El problema de la consistencia en lógica proposicional
- El teorema de Cook
- Variantes de SAT
- Problemas de conjuntos
- Problemas de grafos
- Problemas numéricos
- Otros problemas
- Caracterización de problemas en NP
- Técnicas de reducción de problemas
- Clase CoNP
- Problemas de funciones FNP

- Recordemos que un problema computacional de decisión $P(x)$ es equivalente a un lenguaje:

$$\{x \in A^* : P(x) = 'SI'\}$$

- Y que un lenguaje L sobre A^* define un problema:

Dada $x \in A^*$, ¿Pertenece x a L ?

- La teoría se suele explicar en términos de lenguajes y la práctica en términos de problemas, PERTENECER(x):
- Toda definición en términos de lenguajes tiene una definición equivalente en términos de problemas de decisión y recíprocamente.

Reducción en términos de problemas

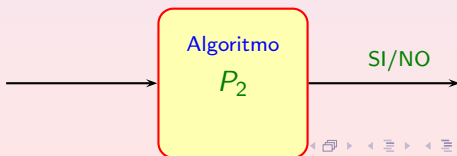
Un problema de decisión $P_1(x)$ es **reducible** a un problema de decisión $P_2(y)$ donde $x \in X, y \in Y$,

$$P_1(x) \propto P_2(y)$$

si y solo si existe un algoritmo determinista que en **espacio logarítmico** calcula una función $REDU : X \rightarrow Y$ de tal manera que

$$P_1(x) = P_2(REDU(x))$$

Problema $P_1(x)$ **reducible a** $P_2(y)$: Algoritmo $P_2 \rightarrow$ Algoritmo P_1



Reducción en términos de problemas

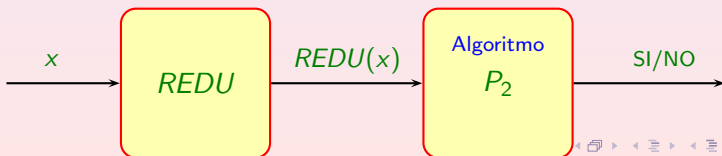
Un problema de decisión $P_1(x)$ es **reducible** a un problema de decisión $P_2(y)$ donde $x \in X, y \in Y$,

$$P_1(x) \propto P_2(y)$$

si y solo si existe un algoritmo determinista que en **espacio logarítmico** calcula una función $REDU : X \rightarrow Y$ de tal manera que

$$P_1(x) = P_2(REDU(x))$$

Problema $P_1(x)$ **reducible a** $P_2(y)$: Algoritmo $P_2 \rightarrow$ Algoritmo P_1



Reducción en términos de problemas

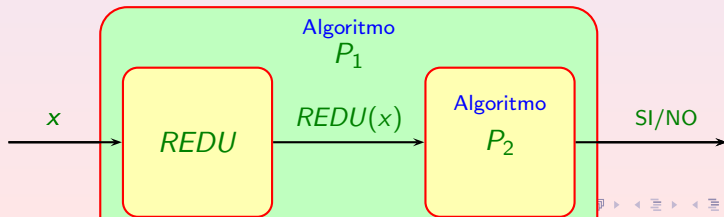
Un problema de decisión $P_1(x)$ es **reducible** a un problema de decisión $P_2(y)$ donde $x \in X, y \in Y$,

$$P_1(x) \propto P_2(y)$$

si y solo si existe un algoritmo determinista que en **espacio logarítmico** calcula una función $REDU : X \rightarrow Y$ de tal manera que

$$P_1(x) = P_2(REDU(x))$$

Problema $P_1(x)$ **reducible a** $P_2(y)$: Algoritmo $P_2 \rightarrow$ Algoritmo P_1



Significado de la Reducción

La reducción de P_1 a P_2 , indica que si obtuviésemos una solución *sencilla* para P_2 , entonces componiéndola con la reducción, podríamos obtener una solución para P_1 .



Eso significa que si obtuviésemos una solución *sencilla* para P_2 la tendríamos para P_1 , pero lo contrario no tiene por qué verificarse.



En definitiva, P_2 es *al menos tan difícil* como P_1 : se resiste más a que se encuentre una solución *sencilla*.

Es una forma de **comparar** problemas.

Reducciones Espacio Logarítmicas

- Nosotros hemos supuesto que las reducciones han de ser espacio logarítmicas.
- En otros textos se supone que son en tiempo polinómico.
- La idea es que la reducción (transformación entre problemas) tiene que poder realizarse de forma "eficiente".
- En realidad las reducciones que aparecen en los textos que suponen que las reducciones se pueden hacer en tiempo polinómico son también espacio logarítmicas.
- Para nosotros la reducción tiene que poder realizarse de forma "muy rápida", que lo concretamos como espacio logarítmico.

Pasos en la Reducción de Problemas

Si tenemos dos problemas $P_1(x)$ y $P_2(y)$ para demostrar que $P_1(x) \propto P_2(y)$ hay que:

- 1 Dar un algoritmo *REDU* que transforme cada entrada, x , de P_1 en una entrada, $y = \text{REDU}(x)$, del problema P_2 .
- 2 Comprobar que dicho algoritmo es logarítmico en espacio.
- 3 Comprobar que si $P_1(x)$ tiene solución positiva, entonces $P_2(y)$ también la tiene.
- 4 Comprobar que si $P_2(y)$ tiene solución positiva, entonces $P_1(x)$ también la tiene.

Datos:

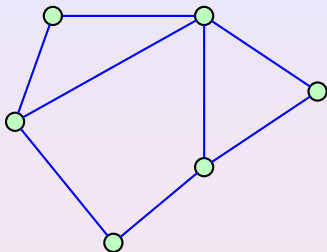
- Un conjunto finito de ciudades $C = \{c_1, \dots, c_m\}$
- Una función de distancia $d : C \times C \rightarrow \mathbb{N}$
- Una cota $B \in \mathbb{N}$

Pregunta: ¿Existe un circuito que visite todas las ciudades una sola vez y de coste no superior a B ?

Es decir, determinar si existe un orden de las ciudades $(c_{\pi(1)}, \dots, c_{\pi(n)})$ tal que

$$\left(\sum_{i=1}^{m-1} d(c_{\pi(i)}, c_{\pi(i+1)}) \right) + d(c_{\pi(m)}, c_{\pi(1)}) \leq B$$

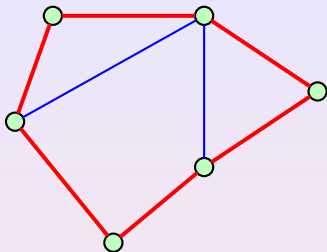
El Problema del Circuito Hamiltoniano (CH)



Datos: Un grafo no dirigido $G = (V, E)$.

Pregunta: ¿Existe un circuito hamiltoniano?

El Problema del Circuito Hamiltoniano (CH)



Datos: Un grafo no dirigido $G = (V, E)$.

Pregunta: ¿Existe un circuito hamiltoniano?

Un circuito hamiltoniano es un camino que parte de un nodo para llegar a él mismo, visitando todos los nodos del grafo una y solo una vez.

Reducción del Circuito Hamiltoniano al problema del Viajante de Comercio: $CH \propto VC$

Supongamos un ejemplo del Circuito Hamiltoniano $G = (V, E)$ con $|V| = m$.

Construimos el siguiente ejemplo del Viajante de Comercio:

- Ciudades $C = V$
- Distancia:

$$d(v_i, v_j) = \begin{cases} 1 & \text{si } (v_i, v_j) \in E \\ 2 & \text{si } (v_i, v_j) \notin E \end{cases}$$

- Cota: $B = m$

Se puede comprobar que la solución del problema del viajante de comercio es siempre mayor o igual a m y que es m si y solo si desde cada ciudad a la siguiente existe un arco en el grafo original.

- La transformación se puede calcular en espacio logarítmico.
- Si existe un circuito hamiltoniano existe un orden de viaje de valor menor o igual a m (el correspondiente al circuito hamiltoniano).
- Si existe un orden de viaje de valor menor o igual a m , entonces todos los arcos tienen coste 1, y dicho circuito es hamiltoniano en el grafo original.

Composición de Reducciones

Si $P_1 \propto P_2$ y $P_2 \propto P_3$, entonces $P_1 \propto P_3$.

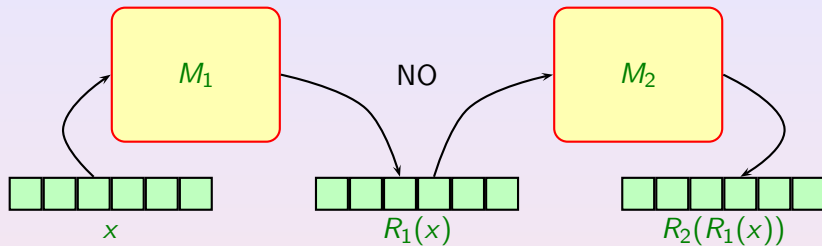
La demostración se basa en la composición de las reducciones de P_1 a P_2 (mediante R_1 calculada por M_1) y de P_2 a P_3 (mediante R_2 calculada por M_2).

La composición no puede ser poner a trabajar M_2 sobre la salida de M_1 : $R_1(x)$: Entonces usaríamos un espacio intermedio polinómico para almacenar $M_1(x)$ y la complejidad de la composición en espacio sería polinómica.

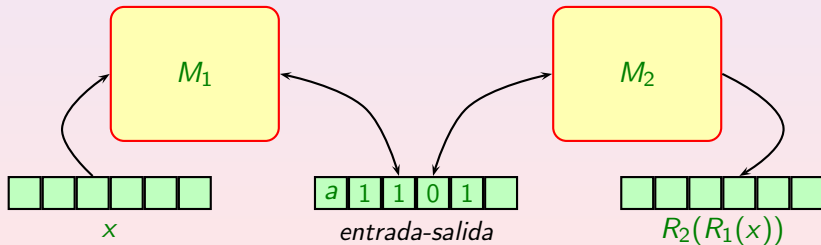
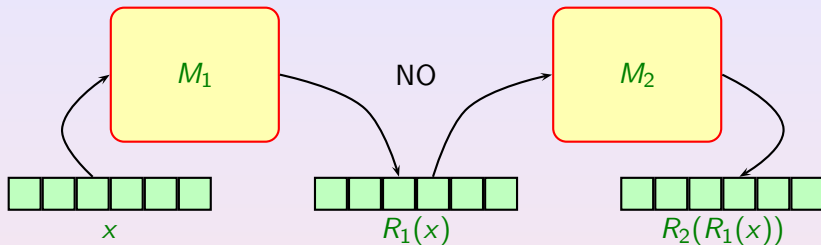
La forma de funcionar es que cada vez que M_2 necesita una casilla se la pide a M_1 que la calcula en ese momento, usando un contador binario para determinar la casilla exacta.

P_1 y P_2 se dicen **equivalentes** si y solo si $P_1 \propto P_2$ y $P_2 \propto P_1$.

Composición en Espacio



Composición en Espacio



Símbolo, Nº Casilla

Problemas NP-completos

Problemas NP-completo

Un problema es **NP-Completo** si y solo si es un problema de NP y cualquier otro problema de NP se reduce a él.

Son dos condiciones la primera es fácil de comprobar, la segunda no.

Los problemas NP-completos son los *más difíciles* o *típicos* dentro de la clase NP.

Dada la definición, 'a priori' no tendría porqué existir ningún lenguaje NP-completo.

Esta definición se extiende a otras clases:

Problemas P-completo

Un problema es **P-Completo** si y solo si es un problema de P y cualquier otro problema de P se reduce a él.

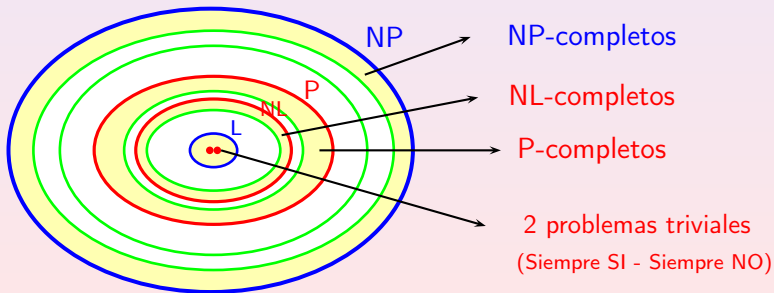
Equivalencia

Dos problemas P_1 y P_2 son **equivalentes** si y solo si

$$P_1 \propto P_2 \text{ y } P_2 \propto P_1$$

Esta es una relación de equivalencia que divide el conjunto de los problemas en clases de equivalencia.

En particular esta relación sobre **NP** queda de la siguiente forma:



Consistencia en Lógica Proposicional (SAT)

Datos: Un conjunto $U = \{p_1, \dots, p_m\}$ de símbolos proposicionales y una colección C de cláusulas sobre estos símbolos.

PREGUNTA: ¿Son consistentes las cláusulas?

EJEMPLO:

- $U = \{p_1, p_2\}$, $C = \{p_1 \vee \neg p_2, \neg p_1 \vee p_2\}$

La respuesta es **SI**, ya que todas las cláusulas se satisfacen haciendo p_1 y p_2 verdaderas. En notación matemática $t(p_1) = t(p_2) = V$.

- $U = \{p_1, p_2\}$, $C = \{p_1 \vee p_2, p_1 \vee \neg p_2, \neg p_1\}$

En este caso, la respuesta es **NO**.

Para satisfacer la tercera cláusula, $t(p_1) = F$ (p_1 es falsa). En estas condiciones, para satisfacer la segunda, tenemos que tener $t(p_2) = F$. Y ahora con p_1 y p_2 falsas, es imposible satisfacer la primera.

Teorema de Cook

El problema de la consistencia en lógica proposicional (SAT) es NP-completo

Demostración:

Lo primero es demostrar que es un problema de NP. Esto es sencillo con el siguiente algoritmo no-determinista:

- Para cada símbolo en U seleccionar un valor de verdad (V ó F).
- Para cada cláusula comprobar si se satisface.
- Si todas se satisfacen responder SI, en caso contrario NO.

Teorema de Cook: Reducción

Ahora hay que demostrar que si un problema, $Pr(x)$, está en NP, entonces dicho lenguaje se reduce a SAT.

Supongamos $Pr(x)$ en NP, la reducción consiste en transformar la entrada al problema x (se suponen una palabra) :
en un ejemplo de SAT tal que

$Pr(x) = \text{'Si'}$ si y solo si el ejemplo es consistente.

Esta transformación tiene que hacerse en espacio logarítmico.

Como $Pr(x)$ está en NP, entonces existe una Máquina de Turing no determinista M que decide $Pr(x)$ en tiempo polinómico (con un polinomio $p(n)$), con un estado de aceptación q_a (para las respuestas positivas) y uno de rechazo q_r (para las respuestas negativas) y una sola cinta (que supondremos ilimitada solo por la derecha).

Supondremos $p(n) \geq n$.

Vamos a construir una transformación que depende de esta máquina:

$$REDU_M : A^* \rightarrow SAT$$

tal que $Pr(x) = 'Si' \Leftrightarrow M \text{ acepta } x \Leftrightarrow REDU_M(x) \text{ es consistente.}$

Teorema de Cook: Notación

- Sea $Q = \{q_0, \dots, q_r\}$, $r = |Q| - 1$, $q_1 = q_a$, $q_2 = q_r$.
- Si $A = \{s_1, \dots, s_v\}$ hagamos $s_0 = \#$ y $B = \{s_0, s_1, \dots, s_v\} = \{\#\} \cup A$.
- Sea l el número máximo de opciones de M (supondremos que siempre hay l opciones excepto en los estados de aceptación y rechazo que no hay ninguna).
- Como la máquina da $p(n)$ pasos, el número máximo de casillas visitadas es $p(n) + 1$.

Teorema de Cook: Variables Proposicionales

Hay cuatro tipos de variables:

- $Q[i, k]$, $0 \leq i \leq p(n)$, $0 \leq k \leq r$
En el momento i , la máquina M está en el estado q_k .
- $H[i, j]$, $0 \leq i \leq p(n)$, $0 \leq j \leq p(n)$
En el momento i , la cabeza está en la casilla j .
- $S[i, j, k]$, $0 \leq i \leq p(n)$, $0 \leq j \leq p(n)$, $0 \leq k \leq v$
En el momento i , la casilla j contiene el símbolo s_k .
- $O(i, k)$, $0 \leq i \leq p(n)$, $1 \leq k \leq l$
En el momento i , la máquina elige la opción k .

Se supone que si la máquina termina en el instante j , todas las variables permanecen inalteradas en los instantes $i > j$ hasta $p(n)$. El número de variables es polinómico y se pueden calcular en espacio logarítmico.

Grupo 1: Estado de la Máquina

$$Q[i, 0] \vee Q[i, 1] \vee \cdots \vee Q[i, r], \quad 0 \leq i \leq p(n)$$

En cada momento, la máquina está en un estado

$$\neg Q[i, j] \vee \neg Q[i, j'], \quad 0 \leq i \leq p(n), \quad 0 \leq j < j' \leq r$$

En cada momento, la máquina no puede estar a la vez en dos estados distintos

$$H[i,0] \vee H[i,1] \vee \cdots \vee H[i,p(n)], \quad 0 \leq i \leq p(n)$$

En cada momento, la cabeza de lectura está situada en alguna casilla

$$\neg H[i,j] \vee \neg H[i,j'], \quad 0 \leq i \leq p(n), \quad 0 \leq j < j' \leq p(n)$$

En cada momento, la cabeza de lectura no puede estar en dos casillas distintas

$$S[i,j,0] \vee S[i,j,1] \vee \cdots \vee S[i,j,v], \quad 0 \leq i \leq p(n), \quad 0 \leq j \leq p(n)$$

En cada momento i , en la casilla j hay algún símbolo del alfabeto.

$$\neg S[i,j,k] \vee \neg S[i,j,k'],$$

$$0 \leq i \leq p(n), \quad 0 \leq j \leq p(n), \quad 0 \leq k < k' \leq v$$

En cada momento i , y en cada casilla j no puede haber dos símbolos distintos.

Grupo 4: Configuración Inicial

- $Q[0,0]$, en el momento inicial (0), la máquina está en el estado 0.
- $H[0,0]$, en el momento inicial (0), la cabeza está en la casilla 0.
- Si la palabra de entrada es: $x = s_{k_1} \dots s_{k_n}$ entonces se introducen las cláusulas:

$$S[0,0,k_1], S[0,1,k_2], \dots, S[0,n-1,k_n]$$

Inicialmente, los símbolos de la palabra x están en las n primeras casillas de la cinta

Estas son las únicas cláusulas que dependen de la entrada.

- $S[0,n,0], S[0,n+1,0], \dots, S[0,p(n),0]$
Inicialmente, el resto de las casillas de entrada contienen el símbolo $B = s_0$.

$$O(i,1) \vee \dots \vee O(i,l), \quad 0 \leq i \leq p(n) - 1.$$

En cada momento la máquina toma una opción.

$$\neg O(i,j) \vee \neg O(i,j'), \quad 0 \leq i \leq p(n) - 1, \quad 1 \leq j, j' \leq l, \quad j \neq j'.$$

La máquina no puede tomar dos opciones a la vez en un momento dado.

Grupo 6: Funcionamiento de la Máquina

Si $\delta(q_k, s_d) = \{(q_{k_1}, s_{d_1}, m_1), \dots, (q_{k_l}, s_{d_l}, m_l)\}$, entonces añadimos todas las cláusulas:

$$\neg H[i, j] \vee \neg Q[i, k] \vee \neg S[i, j, d] \vee \neg O(i, e) \vee Q[i+1, k_e]$$

$$\neg H[i, j] \vee \neg Q[i, k] \vee \neg S[i, j, d] \vee \neg O(i, e) \vee S[i+1, j, d_e]$$

$$\neg H[i, j] \vee \neg Q[i, k] \vee \neg S[i, j, d] \vee \neg O(i, e) \vee H[i+1, j+m_e]$$

donde

$$0 \leq i \leq p(n) - 1, \quad 0 \leq j \leq p(n),$$

$$0 \leq k \leq r, \quad 0 \leq d \leq v, \quad 1 \leq e \leq l$$

$$\neg H[i,j] \vee \neg S[i,j',d] \vee S[i+1,j',d]$$

donde

$$0 \leq i \leq p(n) - 1, \quad 0 \leq j, j' \leq p(n), (j \neq j')$$

$$0 \leq d \leq v$$

Grupo 8: Funcionamiento de la Máquina, después de parar

Si $\delta(q_k, s_d) = \emptyset$, entonces añadimos todas las cláusulas:

$$\neg H[i, j] \vee \neg Q[i, k] \vee \neg S[i, j, d] \vee Q[i + 1, k]$$

$$\neg H[i, j] \vee \neg Q[i, k] \vee \neg S[i, j, d] \vee S[i + 1, j, d]$$

$$\neg H[i, j] \vee \neg Q[i, k] \vee \neg S[i, j, d] \vee H[i + 1, j]$$

donde

$$0 \leq i \leq p(n) - 1, \quad 0 \leq j \leq p(n),$$

$$0 \leq k \leq r, \quad 0 \leq d \leq v, \quad 1 \leq e \leq l$$

Grupo 9: Condición de aceptación

$Q[p(n), 1]$: En el último paso, $p(n)$, la máquina se encuentra en un estado de aceptación.

Equivalencia de los Problemas

Está claro que por la forma de construir la máquina. Las cláusulas reproducen exactamente el funcionamiento de la máquina para la palabra de entrada e incluyen una cláusula que indica que la palabra es aceptada. La libertad está en elegir el valor de verdad de las opciones que puede coger la MT. El resto de las variables tienen un valor de verdad determinado dada la opción elegida. Si al final se puede satisfacer la cláusula $Q[p(n), 1]$ es que se puede llegar a un estado de aceptación. Entonces la palabra es aceptada si y solo si todas es posible satisfacer todas las cláusulas.

Complejidad de la Reducción

La reducción que hemos realizado tiene una complejidad espacio logarítmica en función de la longitud de x : $|x| = n$.

Primero, n se representa con $t = \log(n)$ bits.

El cálculo de $p(n)$ requiere realizar varias operaciones, multiplicaciones y sumas. El número de operaciones es fijo, por lo que el espacio necesario será del orden de $t = \log(n)$.

$p(n)$ se representará también con un número de cifras que será del orden de $t = \log(n)$.

Finalmente, toda la transformación maneja índices que varían entre 0 y $p(n)$ o entre dos valores constantes. En el segundo caso, el espacio no varía en función de la longitud de x , y en el primer caso se necesita un espacio de orden $t = \log(n)$ (el espacio necesario para el índice más grande) para almacenarlos.

Luego, la complejidad en espacio de toda la transformación es de orden $O(\log(n))$.

Estrategia para NP-Compleitud

Supongamos que queremos demostrar que Pr es NP-completo. Tenemos que hacer:

- Demostrar que Pr está en NP.
- Determinar un problema NP-completo ya conocido Pr' , y demostrar que

$$Pr' \propto Pr$$

Como la reducción es transitiva, esta última condición garantiza que cualquier otro problema de NP se reduce a L .

- Hay dos formas de entender 3-SAT: a lo más 3 literales y exactamente 3 literales.
- Las dos son NP-completas.
- Es NP ya que es un caso particular de SAT y éste ya estaba en NP: el mismo algoritmo vale.
- Para demostrar que es completo para la clase NP, no realizamos una demostración similar a la del teorema de Cook. Simplemente, reducimos SAT a 3-SAT.

Para ello, dado un ejemplo de SAT tenemos que construir un ejemplo de 3-SAT que tenga la misma respuesta: las cláusulas son consistentes o inconsistentes en ambos casos a la vez.

SAT $\propto$ 3-SAT

Supongamos un ejemplo de SAT, con símbolos U y un conjunto de cláusulas C .

Construimos un ejemplo de 3-SAT, que tiene como símbolos los de U y algunos adicionales y cuyas cláusulas, C' , se obtienen de C de la siguiente forma:

- Las cláusulas de C de longitud menor o igual a 3 se añaden tal cual a C' .

SAT $\propto$ 3-SAT (Cont.)

La reducción se basa en el siguiente hecho:

Si tengo la fórmula $P \vee Q$ donde P y Q son dos fórmulas, entonces esta fórmula se puede satisfacer si y solo si se puede satisfacer

$$P \vee x, \quad Q \vee \neg x$$

donde x es un símbolo proposicional que no aparecía en la fórmula.

SAT $\propto$ 3-SAT (Cont.)

La reducción se basa en el siguiente hecho:

Si tengo la fórmula $P \vee Q$ donde P y Q son dos fórmulas, entonces esta fórmula se puede satisfacer si y solo si se puede satisfacer

$$P \vee x, \quad Q \vee \neg x$$

donde x es un símbolo proposicional que no aparecía en la fórmula. Si estas dos fórmulas son ciertas entonces haciendo resolución en x , deduzco que $P \vee Q$ se puede satisfacer.

Si $P \vee Q$ se puede satisfacer, entonces existe una asignación con la que P o Q es cierta. Entonces si P es cierta, hago x falso y en caso contrario hago x verdadero: está claro que ambas cláusulas se pueden satisfacer.

SAT $\propto$ 3-SAT (Cont.)

- Para cada cláusula en C de longitud mayor o igual que 4:
 $l_1 \vee l_2 \vee \dots \vee l_k$, $k \geq 4$, donde $l_1, \dots, l_k$ son literales, añado
símbolos proposicionales $x_2, \dots, x_{k-2}$ y las cláusulas

$$l_1 \vee l_2 \vee x_2, \quad \neg x_2 \vee l_3 \vee x_3, \quad \dots, \quad \neg x_{k-3} \vee l_{k-2} \vee x_{k-2}, \quad \neg x_{k-2} \vee l_{k-1} \vee l_k,$$

La demostración de que esta transformación es una verdadera reducción, se basa en comprobar que para cada cláusula $l_1 \vee l_2 \vee \dots \vee l_k$, este es el resultado de aplicar la regla básica $k - 3$ veces con las variables $x_2, \dots, x_{k-2}$.
La primera se aplica, añadiendo x_2 y transformando en

$$l_1 \vee l_2 \vee x_2, \quad \neg x_2 \vee l_3 \vee l_4 \vee \dots \vee l_k$$

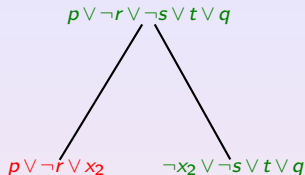
Ahora se aplica recursiva la misma operación a $\neg x_2 \vee l_3 \vee l_4 \vee \dots \vee l_k$, pero añadiendo cada vez una variable distinta x_k , hasta que nos quedemos con una cláusula de longitud 3.

Ejemplo

$$p \vee \neg r \vee \neg s \vee t \vee q$$

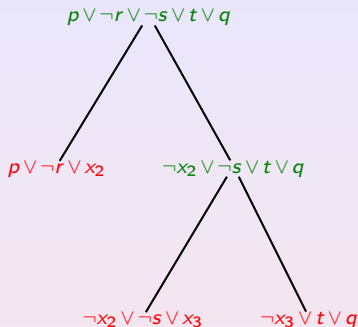
Es importante señalar que para que el algoritmo sea espacio logarítmico, no se puede hacer de forma recursiva calculando las cláusulas intermedias (esas cláusulas pueden ser de tamaño del mismo orden que la entrada). Hay que hacerlo de forma iterativa como se explicó al principio.

Ejemplo



Es importante señalar que para que el algoritmo sea espacio logarítmico, no se puede hacer de forma recursiva calculando las cláusulas intermedias (esas cláusulas pueden ser de tamaño del mismo orden que la entrada). Hay que hacerlo de forma iterativa como se explicó al principio.

Ejemplo



Es importante señalar que para que el algoritmo sea espacio logarítmico, no se puede hacer de forma recursiva calculando las cláusulas intermedias (esas cláusulas pueden ser de tamaño del mismo orden que la entrada). Hay que hacerlo de forma iterativa como se explicó al principio.

Resultado

El problema 3-SAT sigue siendo NP-completo si exigimos que todas las cláusulas tengan exactamente 3 literales distintos.

3-SAT podemos reducirlo a este problema, añadiendo x_1, x_2 y x_3 y añadiendo a toda cláusula de longitud menor que 3, alguna o varias de estas variables.

Por ejemplo $\neg t$ se transformaría en $\neg t \vee x_1 \vee x_2$.

A continuación añadimos cláusulas que fuercen a estas variables a ser falsas: todas las cláusulas de tamaño 3 con estas 3 variables, excepto $x_1 \vee x_2 \vee x_3$. Estas 7 cláusulas solo se pueden satisfacer si x_1, x_2, x_3 son falsas y las nuevas cláusulas son equivalentes a las primeras.

2-SAT está en NL y, por tanto, en P

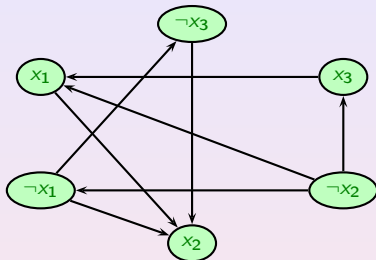
Consideremos un problema 2-SAT, entonces construimos el grafo:

- Nodos: Variables y sus negaciones (todos los posibles literales)
- Arcos: (α, β) es un arco si y solo si $(\neg\alpha \vee \beta)$ está en el problema.

Ejemplo

$$C = \{x_1 \vee x_2, x_1 \vee \neg x_3, \neg x_1 \vee x_2, x_2 \vee x_3\}$$

Grafo:



Teorema

El conjunto de cláusulas C es consistente si y solo si no existe un par de nodos $x, \neg x$, tal que existe un camino de x a $\neg x$ y otro camino de $\neg x$ a x .

Ejemplo

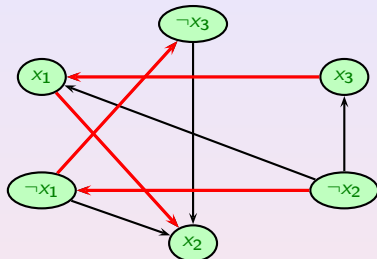
$$C = \{x_1 \vee x_2, x_1 \vee \neg x_3, \neg x_1 \vee x_2, x_2 \vee x_3\}$$

Grafo:

$$x_1 \vee \neg x_3$$

$$\neg x_1 \vee x_2$$

$$\neg x_3 \vee x_2$$



Las deducciones son los caminos.

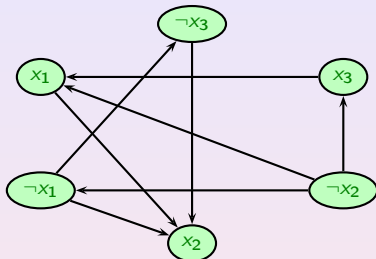
Teorema

El conjunto de cláusulas C es consistente si y solo si no existe un par de nodos $x, \neg x$, tal que existe un camino de x a $\neg x$ y otro camino de $\neg x$ a x .

Ejemplo

$$C = \{x_1 \vee x_2, x_1 \vee \neg x_3, \neg x_1 \vee x_2, x_2 \vee x_3\}$$

Grafo:



Las deducciones son los caminos.

Si hay un camino de x a $\neg x$, se deduce $\neg x$

Si hay un camino de $\neg x$ a x , se deduce x

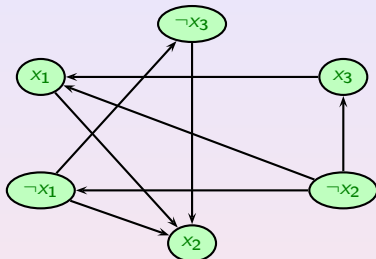
Teorema

El conjunto de cláusulas C es consistente si y solo si no existe un par de nodos $x, \neg x$, tal que existe un camino de x a $\neg x$ y otro camino de $\neg x$ a x .

Ejemplo

$$C = \{x_1 \vee x_2, x_1 \vee \neg x_3, \neg x_1 \vee x_2, x_2 \vee x_3\}$$

Grafo:



Esto demuestra que el complementario de 2-SAT está en NL, pero como $NL=CoNL$, 2-SAT está en NL

Las deducciones son los caminos.

Si hay un camino de x a $\neg x$, se deduce $\neg x$

Si hay un camino de $\neg x$ a x , se deduce x

Teorema

El conjunto de cláusulas C es consistente si y solo si no existe un par de nodos $x, \neg x$, tal que existe un camino de x a $\neg x$ y otro camino de $\neg x$ a x .

Son aquellas en las que, a lo más, existe un literal positivo.

Idea del algoritmo polinómico: (V el conjunto de variables proposicionales).

- Se consideran los conjuntos C_1 (todos los literales son negativos) y C_2 (existe un literal positivo).
- Sea H un conjunto de variables proposicionales que inicialmente es igual a las variables que aparecen en las cláusulas de longitud 1 de C_2 . La idea es que este conjunto contenga las variables tienen que ser necesariamente verdad.

- Mientras H cambie, examinar las cláusulas de C_2 , si alguna es de la forma $\neg z_1 \vee \neg z_2 \vee \dots \vee \neg z_i \vee y$ con todas las variables $z_1, \dots, z_i$ en el conjunto H y la variable y no en H , entonces añadir y a H . (Como $\neg z_1 \vee \neg z_2 \vee \dots \vee \neg z_i \vee y$ es equivalente a $(z_1 \wedge z_2 \wedge \dots \wedge z_i) \rightarrow y$, si todos los antecedentes son verdaderos, entonces el consecuente también lo tiene que ser).
- Una vez calculado H se hacen verdad las variables de este conjunto y falsas las de $V \setminus H$.
- La consistencia es equivalente a que para cada cláusula en C_1 exista un literal con su variable en $V \setminus H$.

Datos:

Un conjunto de cláusulas con dos literales y un valor $K \geq 0$.

Pregunta:

¿Pueden satisfacerse, al menos, K cláusulas?

Es una generalización de 2-SAT y es NP-completo.

Que es NP es inmediato. Para demostrar que es completo en esta clase, vamos a reducir 3-SAT.

Reducción: 3-SAT $\propto$ MAX2SAT

Para ello cada cláusula de longitud 3, $x \vee y \vee z$ se transforma en las siguientes 10 cláusulas (w es una nueva variable añadida),

$$\begin{aligned} &x, \quad y, \quad z, \quad w, \quad \neg x \vee \neg y, \quad \neg y \vee \neg z, \quad \neg z \vee \neg x, \\ &x \vee \neg w, \quad y \vee \neg w, \quad z \vee \neg w \end{aligned}$$

Se selecciona $K = 7m$, donde m es el número de cláusulas.

Técnica: *Construcción de Gadgets*

Esto se puede comprobar, viendo que para cada cláusula, si $x \vee y \vee z$ es verdadero entonces se puede elegir el valor de verdad de w de manera que se satisfagan 7 cláusulas (pero no más). Si $x \vee y \vee z$ es falso, entonces nunca podemos llegar a satisfacer 7 o más de estas 10 cláusulas, elegimos como elijamos el valor de verdad de w .

Muchos problemas, cuando se plantean con suficiente generalidad son NP-completos. Hay restricciones que no llegan a convertirlos en problemas polinómicos, pero siempre, si se restringe el número de casos a resolver lo suficiente se llega un problema polinómico.

Restricción de 3-SAT

3-SAT es NP-completo si hacemos que cada literal nunca aparezca más de dos veces y cada variable más de tres veces, pero pueden existir cláusulas de longitud menor que 3.

Hemos de probar que cualquier ejemplo de 3-SAT se puede transformar en un ejemplo equivalente que cumpla esta restricción. Esto se hace con el siguiente procedimiento:

- Si una variable x no cumple las condiciones de la restricción, apareciendo k veces en las cláusulas, se substituye x por k nuevas variables, $x_1, \dots, x_k$, una por cada aparición.
- Se añaden las cláusulas: $\neg x_1 \vee x_2, \neg x_2 \vee x_3, \dots, \neg x_k \vee x_1$
Estas son equivalentes a: $x_1 \rightarrow x_2, x_2 \rightarrow x_3, \dots, x_k \rightarrow x_1$
con lo que todas las versiones x_i tienen que tener el mismo valor de verdad.

Dado un conjunto de clausulas de longitud 3, determinar si existe una asignación de valores de verdad tal que para cada clausula, alguno, pero no todos los literales sean ciertos.

Es claramente **NP** ya que si se se pueden asignar de forma no determinista valores de verdad a las variables y después comprobar en tiempo polinómico si se verifican las condiciones especificadas: en cada cláusula hay un literal cierto y otro falso.

Reducción de 3-SAT a NAESAT

Se añade una nueva variable z .

Para cada clausula con menos de 3 literales, añadimos z .

Si los tres literales son distintos $p \vee q \vee r$ añadimos una variable s (significado: $p \vee q \leftrightarrow s$) y las cláusulas:

$$s \vee r \vee z, \quad p \vee q \vee \neg s, \quad \neg p \vee s \vee z, \quad \neg q \vee s \vee z$$

Si NAESAT tiene solución, entonces cambiando el valor de verdad de todas las variables, sigue teniendo solución. Elegimos la solución en la que z es falso. Esta es una solución del problema original.

Recíprocamente, si se satisface SAT, podemos satisfacer todas las cláusulas nuevas de acuerdo con NAESAT con el mismo valor de verdad para las variables que existían y haciendo z falso y s verdadero si p ó q son verdaderos.

Isomorfismo de Grafos

Problema del Isomorfismo de Grafos

Dados dos grafos no dirigidos $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$, determinar si existe un isomorfismo entre G_1 y G_2 , esto es una aplicación biyectiva $f : V_1 \rightarrow V_2$, tal que $(u, v) \in E_1 \Leftrightarrow (f(u), f(v)) \in E_2$.

Este es un ejemplo de problema que no se ha demostrado que sea **NP-completo** y tampoco se conoce ningún algoritmo polinómico para resolverlo. Se supone que ni está en **P** ni es **NP-Completo**. De hecho se ha definido una clase **GI** de todos los problemas que se pueden reducir al problema del isomorfismo de grafos, de la que evidentemente el problema del isomorfismo de grafos es **GI-completo**.

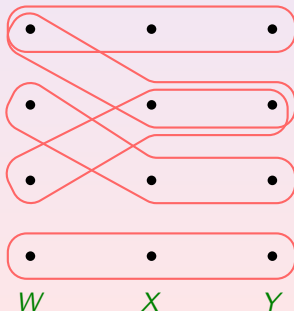
Acoplamiento de Tripletas (ACTRI)

Datos:

- Tres conjuntos disjuntos W, X, Y del mismo tamaño q
- Un subconjunto $M \subseteq W \times X \times Y$ de compatibilidades

Pregunta:

¿Contiene M un subconjunto $M' \subseteq M$ con q elementos, tal que para cada $(w_1, x_1, y_1), (w_2, x_2, y_2) \in M'$, si $(w_1, x_1, y_1) \neq (w_2, x_2, y_2)$, entonces $w_1 \neq w_2, x_1 \neq x_2, y_1 \neq y_2$?



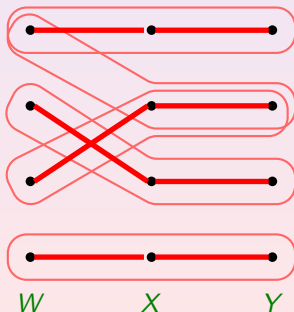
Acoplamiento de Tripletas (ACTRI)

Datos:

- Tres conjuntos disjuntos W, X, Y del mismo tamaño q
- Un subconjunto $M \subseteq W \times X \times Y$ de compatibilidades

Pregunta:

¿Contiene M un subconjunto $M' \subseteq M$ con q elementos, tal que para cada $(w_1, x_1, y_1), (w_2, x_2, y_2) \in M'$, si $(w_1, x_1, y_1) \neq (w_2, x_2, y_2)$, entonces $w_1 \neq w_2, x_1 \neq x_2, y_1 \neq y_2$?



ACTRI es NP-completo

Es NP, ya que un algoritmo no determinista que elige un subconjunto de M y comprueba que si hay en cada elemento del subconjunto uno y sólo uno de los elementos de cada conjunto tiene tiempo polinómico.

Reducción de 3-SAT a ACTRI

Para demostrar que es completo para NP, para a reducir 3-SAT a ACTRI.

Vamos a suponer un ejemplo de 3-SAT con los siguientes datos:

$$U = \{u_1, u_2, \dots, u_n\}, \quad C = \{c_1, c_2, \dots, c_m\}$$

Vamos a construir un ejemplo de ACTRI con la misma solución.

Construiremos los conjuntos W, X, Y y el subconjunto de tripletas

$$M \subseteq W \times X \times Y.$$

Para cada variable u_i vamos a introducir en W los elementos

$u_i[j], \bar{u}_i[j], \quad j = 1, \dots, m$, en X los elementos $a_i[j], j = 1, \dots, m$ y en Y los elementos $b_i[j], j = 1, \dots, m$.

En M se introducen las tripletas: $T_i^t = \{(\bar{u}_i[j], a_i[j], b_i[j]) : 1 \leq j \leq m\}$ y $T_i^f = \{(u_i[j], a_i[j+1], b_i[j]) : 1 \leq j < m\} \cup \{(u_i[m], a_i[1], b_i[m])\}$.

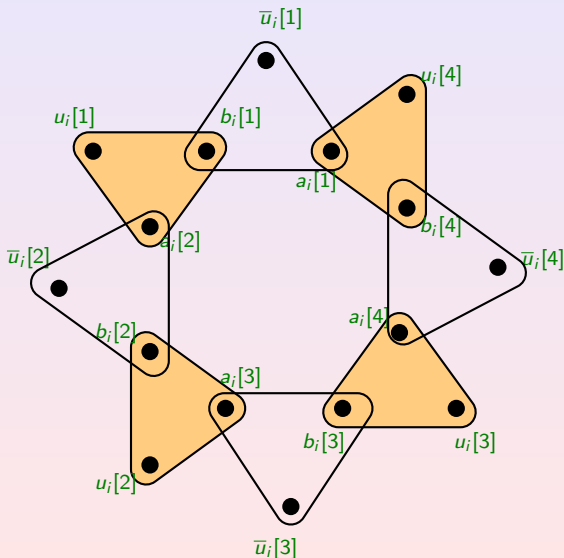
Ejemplo:

$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}$$

$$\{(\bar{u}_1[1], a_1[1], b_1[1]), (\bar{u}_1[2], a_1[2], b_1[2]), (u_1[1], a_1[2], b_1[1]), (u_1[2], a_1[1], b_1[2]), \\ (\bar{u}_2[1], a_2[1], b_2[1]), (\bar{u}_2[2], a_2[2], b_2[2]), (u_2[1], a_2[2], b_2[1]), (u_2[2], a_2[1], b_2[2]), \\ (\bar{u}_3[1], a_3[1], b_3[1]), (\bar{u}_3[2], a_3[2], b_3[2]), (u_3[1], a_3[2], b_3[1]), (u_3[2], a_3[1], b_3[2]), \\ (\bar{u}_4[1], a_4[1], b_4[1]), (\bar{u}_4[2], a_4[2], b_4[2]), (u_4[1], a_4[2], b_4[1]), (u_4[2], a_4[1], b_4[2]), \\ \}$$

ACTRI es NP-Completo

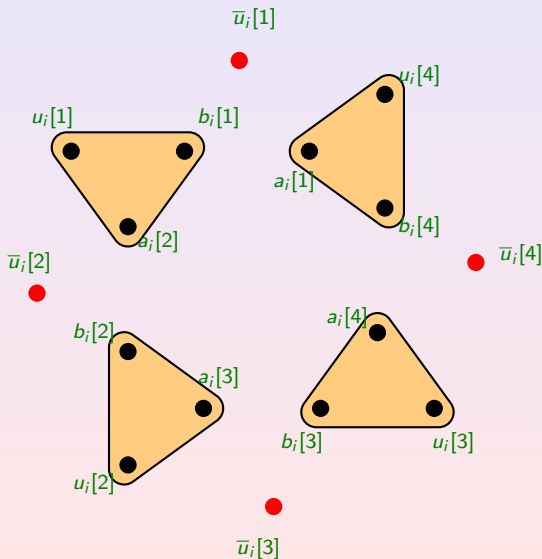
Gráfica de tripletas:



Los valores a_i, b_i no aparecen en otras tripletas. El efecto es que en M' deberán de estar todos los elementos $u_i[1], u_i[2], u_i[3], u_i[4]$ o todos los elementos $\bar{u}_i[1], \bar{u}_i[2], \bar{u}_i[3], \bar{u}_i[4]$.

ACTRI es NP-Completo

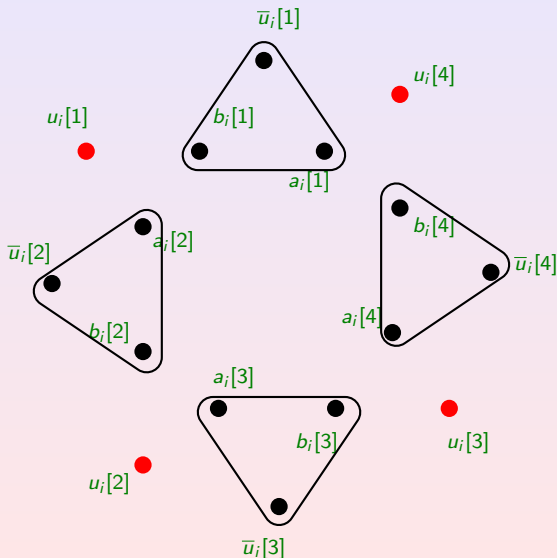
Gráfica de tripletas:



Los valores a_i, b_i no aparecen en otras tripletas. El efecto es que en M' deberán de estar todos los elementos $u_i[1], u_i[2], u_i[3], u_i[4]$ o todos los elementos $\bar{u}_i[1], \bar{u}_i[2], \bar{u}_i[3], \bar{u}_i[4]$.

ACTRI es NP-Completo

Gráfica de tripletas:



Los valores a_i , b_i no aparecen en otras tripletas. El efecto es que en M' deberán de estas todos los elementos $u_i[1], u_i[2], u_i[3], u_i[4]$ o todos los elementos $\bar{u}_i[1], \bar{u}_i[2], \bar{u}_i[3], \bar{u}_i[4]$.

Tripletas para las cláusulas

Por cada cláusula $c_j \in C$:

- Añadir un elemento $s[j]$ a X
- Añadir un elemento $t[j]$ a Y
- Añadir a M las tripletas

$\{(u_i[j], s[j], t[j]) : u_i \text{ está en } c_j\} \cup$

$\{(\bar{u}_i[j], s[j], t[j]) : \neg u_i \text{ está en } c_j\}$

La idea es que en cada acoplamiento M' debe de contener alguna de estas tripletas.

Si, de las tripletas anteriores, quedaban libres (no elegidos) los elementos $u_i[1], u_i[2], u_i[3], u_i[4]$ corresponde al caso en el que u_i es cierto. Si quedaban libres $\bar{u}_i[1], \bar{u}_i[2], \bar{u}_i[3], \bar{u}_i[4]$, corresponde al caso en el que u_i es falso.

En consecuencia, estas tripletas garantizan que todas las cláusulas se satisfacen.

Elementos adicionales

Se añden los siguientes elementos:

- $g[k]$, $1 \leq k \leq m(n-1)$ al conjunto X
- $h[k]$, $1 \leq k \leq m(n-1)$ al conjunto Y
- Al conjunto M las tripletas:

$$(u_i[j], g[k], h[k]), (\bar{u}_i[j], g[k], h[k]), \\ 1 \leq k \leq m(n-1), 1 \leq i \leq n, 1 \leq j \leq m$$

La idea es la siguiente:

Nos han quedado libres en W después de elegir entre las primeras tripletas: $m \cdot n$ elementos. Después de la segundas tripletas, como hemos elegido uno por cláusula, nos quedan $m \cdot n - m = m(n-1)$. Ahora añadimos este número de elementos a X y a Y y le permitimos que sean compatibles con todos los de X , para tener libertad para elegir los elementos sobrantes en tripletas como queramos.

El número de tripletas de M es: $2mn + 3m + 2m^2n(n-1)$

Ejemplo

$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}$$

$$W = \{u_1[1], u_1[2], \bar{u}_1[1], \bar{u}_1[2], u_2[1], u_2[2], \bar{u}_2[1], \bar{u}_2[2], \\ u_3[1], u_3[2], \bar{u}_3[1], \bar{u}_3[2], u_4[1], u_4[2], \bar{u}_4[1], \bar{u}_4[2]\}$$

$$X = \{a_1[1], a_1[2], a_2[1], a_2[2], a_3[1], a_3[2], a_4[1], a_4[2], s[1], s[2], \\ g[1], g[2], g[3], g[4], g[5], g[6]\}$$

$$Y = \{b_1[1], b_1[2], b_2[1], b_2[2], b_3[1], b_3[2], b_4[1], b_4[2], t[1], t[2], \\ h[1], h[2], h[3], h[4], h[5], h[6]\}$$

$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}$$

$M =$

$$\begin{aligned} & \{(\bar{u}_1[1], a_1[1], b_1[1]), (\bar{u}_1[2], a_1[2], b_1[2]), (u_1[1], a_1[2], b_1[1]), (u_1[2], a_1[1], b_1[2]), \\ & (\bar{u}_2[1], a_2[1], b_2[1]), (\bar{u}_2[2], a_2[2], b_2[2]), (u_2[1], a_2[2], b_2[1]), (u_2[2], a_2[1], b_2[2]), \\ & (\bar{u}_3[1], a_3[1], b_3[1]), (\bar{u}_3[2], a_3[2], b_3[2]), (u_3[1], a_3[2], b_3[1]), (u_3[2], a_3[1], b_3[2]), \\ & (\bar{u}_4[1], a_4[1], b_4[1]), (\bar{u}_4[2], a_4[2], b_4[2]), (u_4[1], a_4[2], b_4[1]), (u_4[2], a_4[1], b_4[2]), \\ & (u_1[1], s[1], t[1]), (\bar{u}_3[1], s[1], t[1]), (\bar{u}_4[1], s[1], t[1]), \\ & (\bar{u}_1[2], s[2], t[2]), (u_2[2], s[2], t[2]), (\bar{u}_4[2], s[2], t[2]), \\ & (u_i[j], g[k], h[k]), (\bar{u}_i[j], g[k], h[k]) (i = 1, \dots, 4, j = 1, 2, k = 1, \dots, 6)\} \end{aligned}$$

Solución 3-SAT implica solución ACTRI

Si tenemos una asignación de valores de verdad $t(u_i) \in \{V, F\}$ que satisface todas las cláusulas, elegimos el subconjunto de tripletas M' de la siguiente forma:

- Si u_i es cierto elegimos las tripletas:
 $\{(\bar{u}_i[j], a_i[j], b_i[j]) : 1 \leq j \leq m\}$
- Si u_i es falso:
 $\{(u_i[j], a_i[j+1], b_i[j]) : 1 \leq j < m\} \cup \{(u_i[m], a_i[1], b_i[m])\}.$
- Para cada cláusula j elegimos de entre las tripletas:
 $\{(u_i[j], s[j], t[j]) : u_i \text{ está en } c_j\} \cup$
 $\{(\bar{u}_i[j], s[j], t[j]) : \neg u_i \text{ está en } c_j\}$ elegimos aquella que corresponde a la variable u_i que hacer cierta la cláusula.
- Nos quedan por elegir exactamente $n(m-1)$ elementos de cada conjunto, que se eligen de las tripletas $(u_i[j], g[k], h[k]), (\bar{u}_i[j], g[k], h[k])$. Aquí no hay problema, ya que todos los elementos son compatibles entre sí.

Solución ACTRI implica solución 3-SAT

Supongamos el problema de ACTRI construido a partir de 3-SAT y supongamos que tenemos un subconjunto M' de tripletas solución.

- 1 Vimos que para cada variable u_i , sólo hay dos formas de elegir las tripletas en una solución: o se eligen las $\{(\bar{u}_i[j], a_i[j], b_i[j]) : 1 \leq j \leq m\}$ o se eligen las $\{(u_i[j], a_i[j+1], b_i[j]) : 1 \leq j < m\} \cup \{(u_i[m], a_i[1], b_i[m])\}$.
- 2 En el primer caso, hacemos u_i verdadero y en el segundo u_i falso.
- 3 Para cada cláusula alguna de las 3 tripletas: $\{(u_i[j], s[j], t[j]) : u_i \text{ está en } c_j\} \cup \{(\bar{u}_i[j], s[j], t[j]) : \neg u_i \text{ está en } c_j\}$ debe de haberse elegido en M' .
- 4 Si hemos elegido $(u_i[j], s[j], t[j])$ con u_i en c_j , entonces, $u_i[j]$ no estaba en las tripletas elegidas consideradas en el paso 1. Por lo tanto, hemos hecho u_i verdadero y la cláusula se satisface.
- 5 Si hemos elegido $(\bar{u}_i[j], s[j], t[j])$ con $\neg u_i$ en c_j , entonces, $\bar{u}_i[j]$ no estaba en las tripletas elegidas consideradas en el paso 1. Por lo tanto, hemos hecho u_i falso y la cláusula se satisface.

Cubr. por conj. de tamaño tres (3-SET)

Datos: Un conjunto finito X con $|X| = 3q$ y un subconjunto C de subconjuntos de X de tres elementos.

Pregunta: ¿Existe $C' \subseteq C$ tal que $X = \bigcup_{A \in C'} A$ y los elementos de C' son disjuntos dos a dos ($A, B \in C'$ y $A \neq B$, entonces $A \cap B = \emptyset$)?

Es un problema de NP.

Para demostrar que es completo para NP, existe una transformación muy sencilla: $\text{ACTRI} \propto \text{3-SET}$.

Supongamos un ejemplo de ACTRI dado por W, Y, Z y el subconjunto de tripletas M , construimos el ejemplo de 3-SET donde $X = W \cup Y \cup Z$ y

$$C = \{\{w, y, z\} : (w, y, z) \in M\}$$

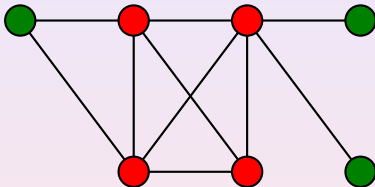
Clique: Dado un grafo $G = (V, E)$, un clique es un subconjunto maximal totalmente conectado. Es decir, un subconjunto $V_t \subseteq V$ tal que $\forall v_1, v_2 \in V_t, (v_1, v_2) \in E$, y que no está estrictamente incluido en otro conjunto que cumpla esta propiedad.

El problema del Clique Máximo (Clique): Dado un grafo $G = (V, E)$ y un número natural $J \leq |V|$, determinar si existe un clique de tamaño mayor o igual que J .

Este problema es equivalente a la existencia de un conjunto totalmente conectado de tamaño mayor o igual que J .

Ejemplo

En el siguiente grafo, tenemos un clique de tamaño 4 (en rojo):



Cubrimiento por Vértices (CV)

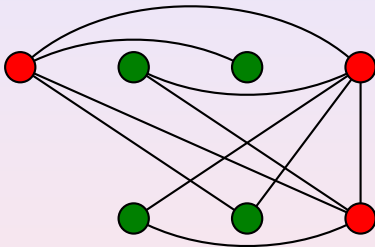
Dado un grafo $G = (V, E)$ y un subconjunto $V_c \subseteq V$, se dice que V_c es un **cubrimiento por vértices** de G , si y solo si toda arista del grafo tiene un extremo en V_c :

$$\forall (u, v) \in E, \quad (u \in V_c \vee v \in V_c)$$

Problema: Dado un grafo $G = (V, E)$ y un número natural $K \leq |V|$, determinar si existe un cubrimiento por vértices de tamaño menor o igual que K .

Ejemplo

En el siguiente grafo, tenemos un cubrimiento por vértices de tamaño 3 (en rojo):



Conjunto Independiente (CI)

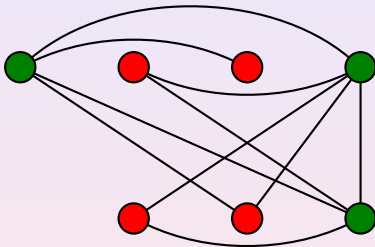
Dado un grafo $G = (V, E)$ y un subconjunto $V_i \subseteq V$, se dice que V_i es un **conjunto independiente** de G , si y solo si no hay ninguna arista que una vértices de V_i :

$$\forall u, v \in V_i, \quad (u, v) \notin E$$

Problema: Dado un grafo $G = (V, E)$ y un número natural $J \leq |V|$, determinar si existe un conjunto independiente de tamaño mayor o igual que J .

Ejemplo

En el siguiente grafo, tenemos un conjunto independiente de tamaño 4 (en rojo):

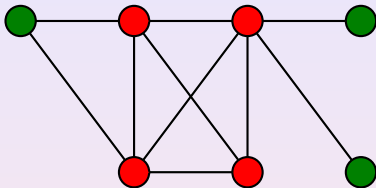


Si $G = (V, E)$ es un grafo y $V^* \subseteq V$, entonces las siguientes condiciones son equivalentes:

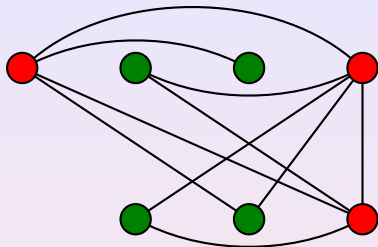
- a) V^* es un cubrimiento por vértices de G
- b) $V \setminus V^*$ es un conjunto independiente de G
- c) $V \setminus V^*$ es un subgrafo totalmente conectado del grafo complementario $\overline{G} = (V, \overline{E})$.
Donde $\overline{E} = V \times V \setminus E$.

Ejemplo

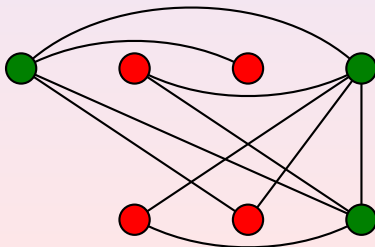
Totalmente Conectado



Cubrimiento por Vértices



Conjunto Independiente



Equivalencia de Problemas

Los tres problemas (Clique, Cubrimiento por Vértices y Conjunto Independiente) son equivalente y si uno es NP-completo los otros también lo son.

Si n es el número de nodos de G , entonces:

G tiene un cubrimiento por vértices de tamaño menor o igual que K



G tiene un conjunto independiente de tamaño mayor o igual que $n - K$



$\overline{G}$ tiene un clique de tamaño mayor o igual que $n - K$

Cubr. por Vértices (CV) es NP-completo

Está claro que es NP: se elige de forma no determinista un subconjunto de vértices y se comprueba en tiempo polinómico si es un cubrimiento.

Para demostrar que es completo reduciremos 3-SAT: $3\text{-SAT} \propto \text{CV}$.
Sea una ejemplo de 3-SAT con variables U y cláusulas C .
Se crea un ejemplo de CV, dado por un grafo $G = (V, E)$.

Elementos del grafo

- Para cada variables $u_i \in U$ se añaden dos vértices $u_i, \bar{u}_i$ y un arco que los una.
- Para cada cláusula $c_j \in C$, se añaden tres vértices $a_1[j], a_2[j], a_3[j]$ y tres aristas que los unan (se forma un triángulo).
- Para cada cláusula $c_j \in C$ si en el literal número k de esta cláusula aparece la variable u_i , se añade una arista de $u_k[j]$ al vértice u_i si la variable aparece positiva y al vértice $\bar{u}_i$ si la variable aparece negada.
- Se pone un límite $K = n + 2m$.

Ejemplo

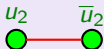
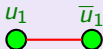
Ejemplo de 3-SAT:

$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$

Ejemplo

Ejemplo de 3-SAT:

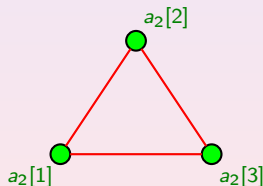
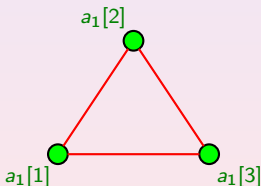
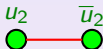
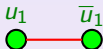
$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$



Ejemplo

Ejemplo de 3-SAT:

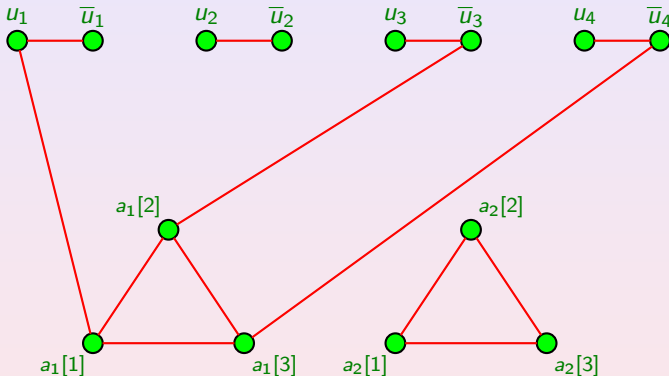
$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$



Ejemplo

Ejemplo de 3-SAT:

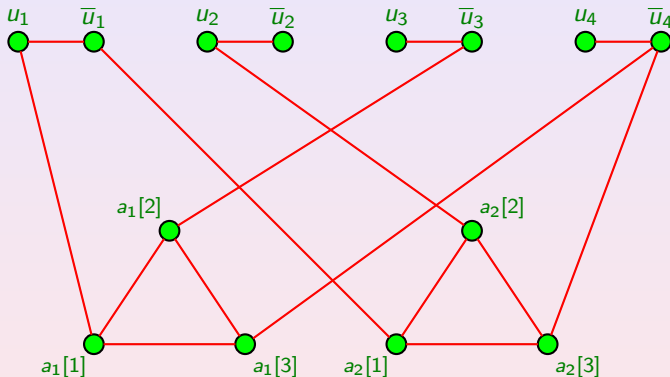
$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$



Ejemplo

Ejemplo de 3-SAT:

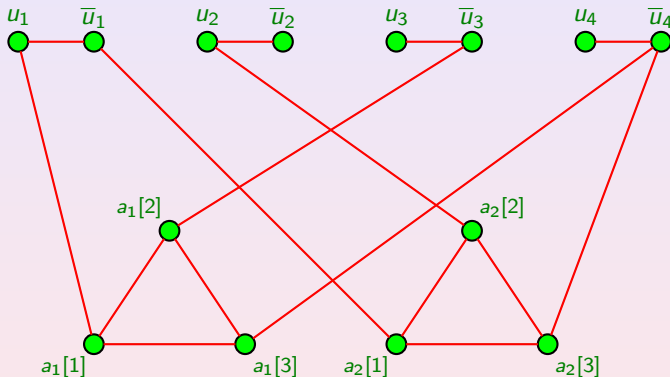
$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$



Ejemplo

Ejemplo de 3-SAT:

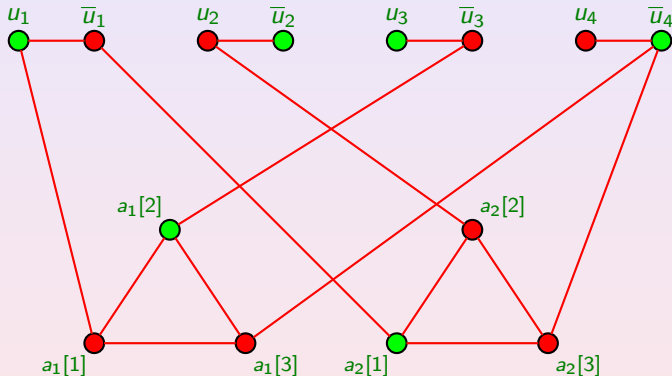
$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$



Equivalencia CV $\leftrightarrow$ 3-SAT

Ejemplo de 3-SAT:

$$U = \{u_1, u_2, u_3, u_4\}, \quad C = \{u_1 \vee \neg u_3 \vee \neg u_4, \neg u_1 \vee u_2 \vee \neg u_4\}.$$



u_1 falso

u_2 verdadero

u_3 falso

u_4 verdadero

Cubrimiento:

Vértices rojos

CV implica 3-SAT

Si existe un cubrimiento V_C , entonces

- Debe de haber, al menos, un vértice de cada pareja, $u_i, \overline{u_i}$
- Para cada cláusula C_j deben de existir, al menos, dos vértice de cada conjunto: $\{a_1[j], a_2[j], a_3[j]\}$.

Como el cubrimiento tiene, a lo más, $n + 2m$ vértices, entonces habrá exactamente un vértice por cada pareja $u_i, \overline{u_i}$ y dos por cada conjunto $\{a_1[j], a_2[j], a_3[j]\}$.

CV implica 3-SAT

Las cláusulas se pueden satisfacer haciendo para cada variable u_i :

- u_i cierto si $u_i \in V_c$
- u_i falso si $u_i \notin V_c$

Cada cláusula C_j tiene tres vértices, y cada vértice está conectado con un vértice de variable.

De estos tres arcos, hay dos que tienen extremos en los dos vértices de $\{a_1[j], a_2[j], a_3[j]\}$ que están en V_c

El otro arco, tendrá que tener su extremo del cubrimiento en los vértices correspondientes a las variables: u_i o $\overline{u_i}$. El valor de verdad asignado a dicha variable hace que esa cláusula se satisfaga.

Sea una asignación de valores de verdad que haga consistentes las cláusulas, entonces un cubrimiento por vértices del tamaño deseado se consigue de la siguiente forma:

$$V_c = \{u_i : u_i \text{ es verdadero}\} \cup \{\overline{u_i} : u_i \text{ es falso}\} \cup \\ (\bigcup_j (\{a_1[j], a_2[j], a_3[j]\} - \{a_i[j]\}))$$

donde $a_i[j]$ corresponde al literal que hace verdadera la cláusula C_j .

El Problema del Circuito Hamiltoniano (CH)

El problema del Circuito Hamiltoniano es NP-completo.

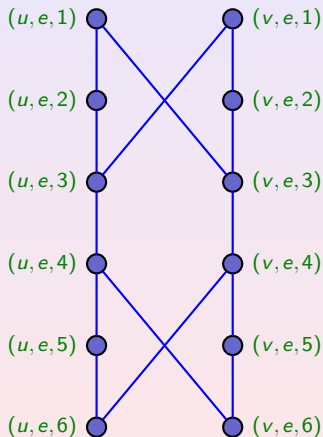
Es inmediato que es NP. El algoritmo no-determinista polinómico solo tiene que elegir n nodos (número de nodos en el grafo) y después comprobar que hay un arco desde cada nodo al siguiente y del último hasta el primero.

Para demostrar que es completo, vamos a reducir CV a este problema. Sea $G = (V, E)$ y $K \leq |V|$ un problema de cubrimiento por vértices.

Vamos a construir un grafo $G' = (V', E')$ de tal forma que la existencia de un circuito hamiltoniano para G' sea equivalente a la existencia de un recubrimiento de tamaño $\leq K$ para G .

Reducción: grafos base

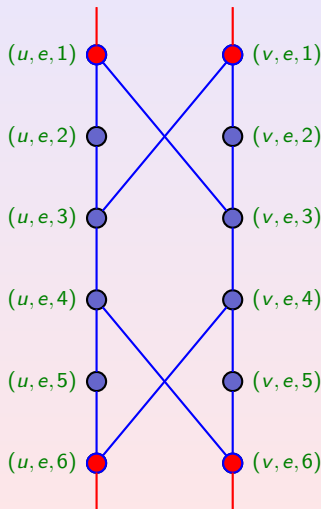
Para cada $e = (u, v) \in E$ se añade el siguiente grafo a G' (con 12 vértices).



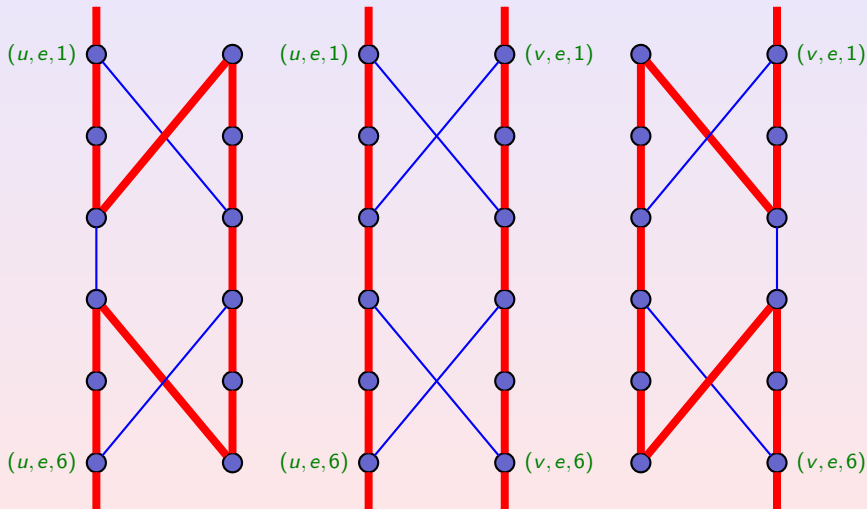
Reducción: grafos base

Para cada $e = (u, v) \in E$ se añade el siguiente grafo a G' (con 12 vértices).

Solo estos nodos de los extremos se conectan con el resto del grafo



Reducción: recorridos



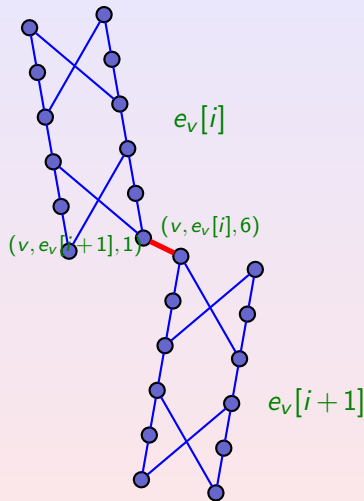
Reducción: conectando grafos básicos

Para cada $v \in V$ sea

$$e_v[1], \dots, e_v[r_v]$$

una ordenación arbitraria de los arcos que contienen v .

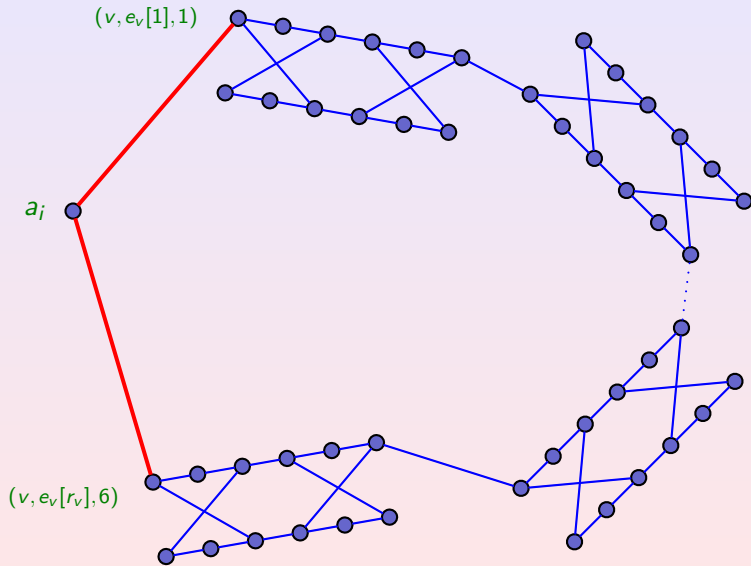
Para $1 \leq i < r_v$ unimos el subgrafo asociado a $e_v[i]$ y el subgrafo $e_v[i+1]$ mediante un arco que va de $(v, e_v[i], 6)$ a $(v, e_v[i+1], 1)$.



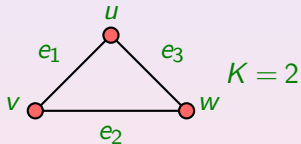
Finalmente se añaden K vértices, $a_1, a_2, \dots, a_K$, que se unen con los subgrafos de la siguiente forma:

- Para cada $v \in V$ sea $e_v[1], \dots, e_v[r_v]$, la lista de las aristas que lo contienen en el orden que se consideraron anteriormente
 - Se añade una arista de cada uno de los a_i a $(v, e_v[1], 1)$
(vértice extremo de la primera arista correspondiente a v)
 - Se añade una arista de cada uno de los a_i a $(v, e_v[r_v], 6)$
(vértice extremo de la última arista correspondiente a v)

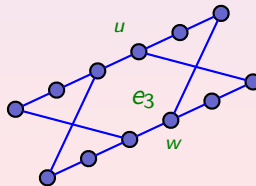
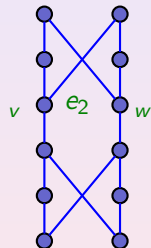
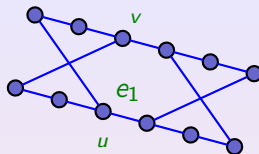
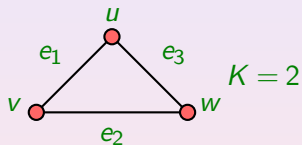
Reducción: Nodos Adicionales



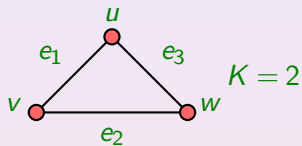
Ejemplo



Ejemplo



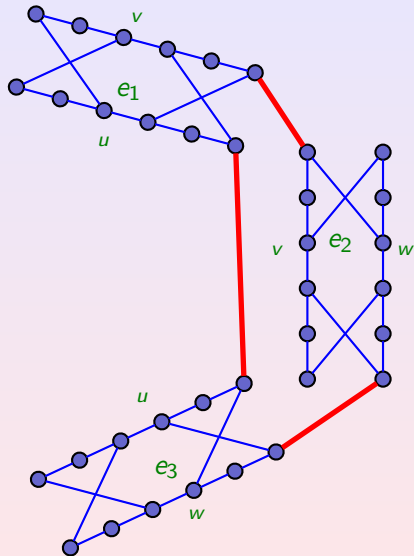
Ejemplo



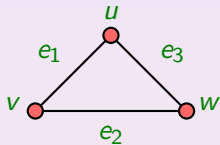
Nodo u : aristas e_1, e_3

Nodo v : aristas e_1, e_2

Nodo w : aristas e_2, e_3



Ejemplo

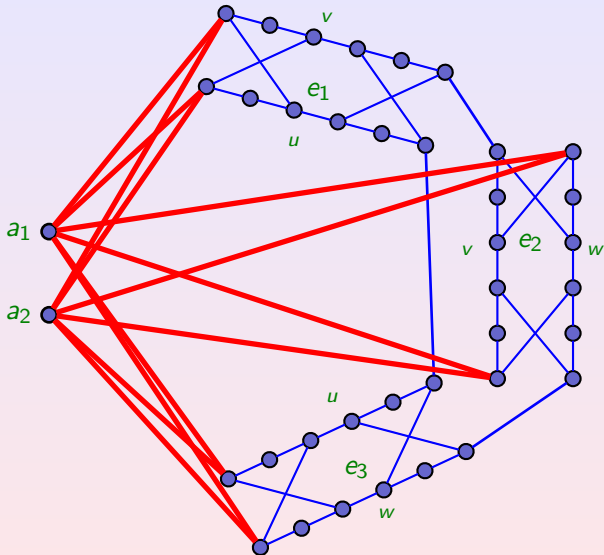


$K = 2$

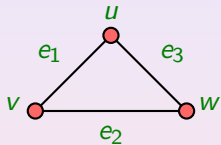
Nodo u : aristas e_1, e_3

Nodo v : aristas e_1, e_2

Nodo w : aristas e_2, e_3



Ejemplo

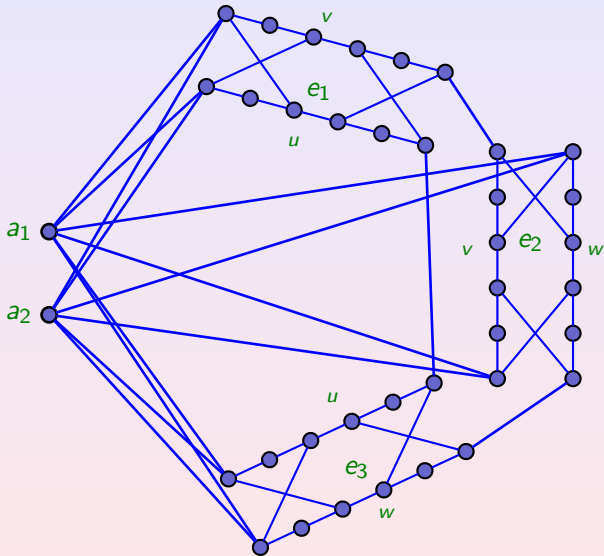


$K = 2$

Nodo u : aristas e_1, e_3

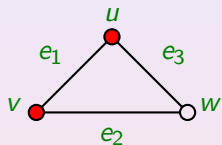
Nodo v : aristas e_1, e_2

Nodo w : aristas e_2, e_3



Equivalencia de Soluciones

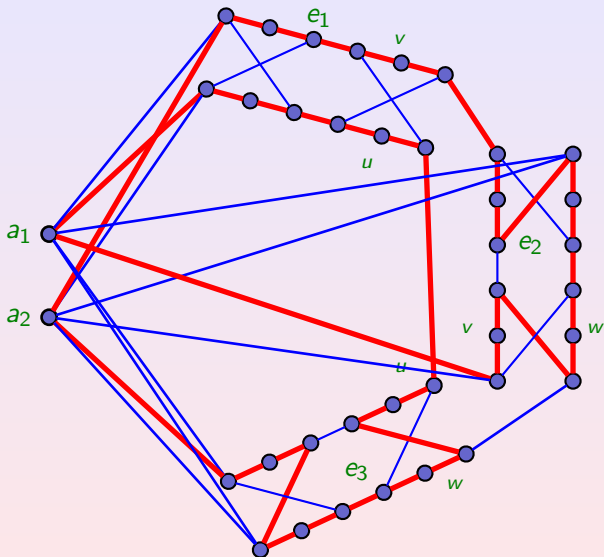
$K = 2$



Nodo u : aristas e_1, e_3

Nodo v : aristas e_1, e_2

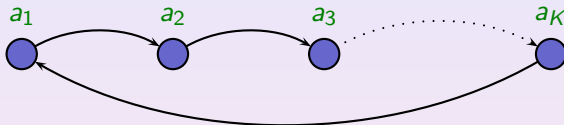
Nodo w : aristas e_2, e_3



Sean K vértices que forman un cubrimiento por vértices: $\{v_1, \dots, v_K\}$. Un circuito hamiltoniano se puede construir de la siguiente forma comenzando en a_1 :

- Si estamos en a_i , desde él recorremos todos los grafos asociados a las aristas de v_i : para cada arista si tiene sólo a v_i en el cubrimiento por vértices, se recorre el subgrafo de esa arista de forma completa; si la arista contiene los dos extremos en el cubrimiento, se recorre sólo la mitad de los vértices del subgrafo correspondientes a v_i . Desde el último subgrafo volvemos a a_{i+1} , repitiendo el proceso, excepto para a_K que volvemos a a_1 y termina el circuito.

Si tenemos un circuito hamiltoniano, tendrá la forma



Desde cada a_i al siguiente a_{i+1} (y también desde a_K a a_1) recorremos los subgrafos. En cada uno de esos recorridos entramos y salimos en los subgrafos por un mismo vértice. Sea este vértice v_i (v_K si es el recorrido de a_K a a_1).

El conjunto $\{v_1, \dots, v_K\}$ es un cubrimiento por vértices, ya que cada arista tiene un subgrafo en el que, al menos, el circuito hamiltoniano entra una vez. El vértice por el que se entra (que ha de coincidir con el de salida) ha de estar en el cubrimiento por vértices.

El Problema de la SUMA

Datos: Un conjunto A y un tamaño para cada uno de sus elementos:

$$s : A \rightarrow \mathbb{N}$$

y un número entero B **Pregunta:** Determinar si existe un $A' \subseteq A$ tal que se verifique:

$$\sum_{a \in A'} s(a) = B$$

Este es claramente un problema de NP: se eligen de forma no determinista los elementos de A' y en tiempo polinómico se determina si la suma de los tamaños de los conjuntos es igual a B . Para demostrar que es completo para NP, vamos a reducir el cubrimiento por tripletas (ACTRI) a este problema.

Sea W, X, Y con $|W| = |X| = |Y| = q$ y un subconjunto $M \subseteq X \times Y \times Z$ un ejemplo del problema ACTRI, vamos a construir una partición equivalente.

Consideremos:

$$W = \{w_1, \dots, w_q\}, \quad X = \{x_1, \dots, x_q\}$$

$$Y = \{y_1, \dots, y_q\}$$

$$M = \{m_1, m_2, \dots, m_k\}$$

El conjunto A va a contener k elementos $A = \{a_1, \dots, a_k\}$.
Cada elemento $a_i \in A$ se corresponde con una tripleta $m_i \in M$.

Pregunta

Si sumamos k unos, ¿cuántas cifras como máximo puede tener el número resultante?

Pregunta

Si sumamos k unos, ¿cuántas cifras como máximo puede tener el número resultante?

$$\begin{array}{r} 1 \\ 1 \\ 1 \\ \dots \\ 1 \\ \hline 1 \ 0 \ 1 \ 1 \ 1 \end{array}$$

El resultado tiene a lo más tamaño $p = \lceil \log_2(k) \rceil + 1$ (sumar uno a $\log_2(k)$ sin decimales).

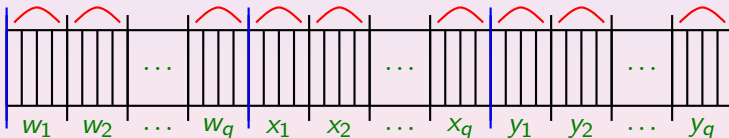
Reducción ACTRI $\propto$ SUMA

Para cada tripleta $m_i = (w_{f(i)}, x_{g(i)}, y_{h(i)})$, se considera el elemento a_i con un peso:

$$s(a_i) = 2^{p(3q-f(i))} + 2^{p(2q-g(i))} + 2^{p(q-h(i))}$$

donde $p = \lceil \log_2(k) \rceil + 1$.

En binario podemos ver el número en grupos de p posiciones. Cada grupo corresponde a un elemento de W, X o Y :



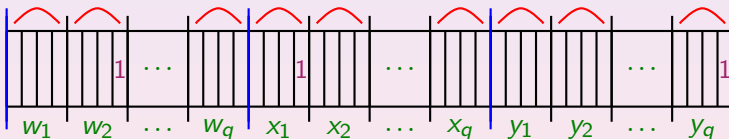
Cada tripleta $m_i = (w_{f(i)}, x_{g(i)}, y_{h(i)})$ se corresponde con un número con un **1** a la derecha de las zonas de $w_{f(i)}, x_{g(i)}, y_{h(i)}$ y **0** en el resto.

Reducción ACTRI $\propto$ SUMA

Para cada tripleta $m_i = (w_{f(i)}, x_{g(i)}, y_{h(i)})$, se considera el elemento a_i con un peso:

$$s(a_i) = 2^{p(3q-f(i))} + 2^{p(2q-g(i))} + 2^{p(q-h(i))}$$

donde $p = \lceil \log_2(k) \rceil + 1$.
En binario podemos ver el número en grupos de p posiciones. Cada grupo corresponde a un elemento de W, X o Y :



Cada tripleta $m_i = (w_{f(i)}, x_{g(i)}, y_{h(i)})$ se corresponde con un número con un 1 a la derecha de las zonas de $w_{f(i)}, x_{g(i)}, y_{h(i)}$ y 0 en el resto.

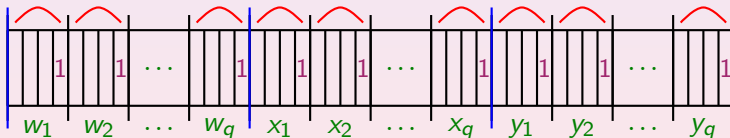
Ejemplo: Tripleta: (w_2, x_1, y_q)

Reducción $ACTRI \propto SUMA$

Si $p = \lceil \log_2(k) \rceil + 1$, entonces sumando k o menos unos nunca obtenemos un número con un 1 más allá de la posición $p+1$

Sumando k o menos números de los asociados a las tripletas, nunca pasa un 1 de la zona de un elemento a la zona de otro.

Finalmente, $B = \sum_{j=0}^{3q-1} 2^{p \cdot j}$. Este número tiene un 1 a la derecha de cada una de las zonas:



Ejemplo

Supongamos el problema de tripletas con:

$W = \{w_1, w_2, w_3\}$, $X = \{x_1, x_2, x_3\}$, $Y = \{y_1, y_2, y_3\}$ y las tripletas $(w_2, x_1, y_3), (w_2, x_2, y_2), (w_3, x_2, y_1), (w_3, x_3, y_2), (w_1, x_3, y_2)$.

Ejemplo

Supongamos el problema de tripletas con:

$W = \{w_1, w_2, w_3\}$, $X = \{x_1, x_2, x_3\}$, $Y = \{y_1, y_2, y_3\}$ y las tripletas (w_2, x_1, y_3) , (w_2, x_2, y_2) , (w_3, x_2, y_1) , (w_3, x_3, y_2) , (w_1, x_3, y_2) .

Construimos el siguiente problema de la SUMA: el número de tripletas es $k = 5$, y $p = \lceil \log(k) \rceil + 1 = 3$. Hay un individuo para cada tripleta definido por su peso (se supone un orden $w_1, w_2, w_3, x_1, x_2, x_3, y_1, y_2, y_3$):

	w_1	w_2	w_3	x_1	x_2	x_3	y_1	y_2	y_3	
1	000	001	000	001	000	000	000	000	001	(w_2, x_1, y_3)
2	000	001	000	000	001	000	000	001	000	(w_2, x_2, y_2)
3	000	000	001	000	001	000	001	000	000	(w_3, x_2, y_1)
4	000	000	001	000	000	001	000	001	000	(w_3, x_3, y_2)
5	001	000	000	000	000	001	000	001	000	(w_1, x_3, y_2)

y $B = 001\ 001\ 001\ 001\ 001\ 001\ 001\ 001\ 001\ 001$.

Equivalencia de las soluciones

Las tripletas 1, 3, 5: $(w_2, x_1, y_3), (w_3, x_2, y_1), (w_1, x_3, y_2)$ son una solución del problema original y los individuos correspondientes a estas tripletas suman B .

1	000 001 000 001 000 000 000 000 001	(w_2, x_1, y_3)
3	000 000 001 000 001 000 001 000 000	(w_3, x_2, y_1)
5	001 000 000 000 000 000 001 000 001 000	(w_1, x_3, y_2)
B	001 001 001 001 001 001 001 001 001 001	

Como una consencuencia, para cada $A' \subseteq \{a_1, \dots, a_k\}$, tenemos que

$$\sum_{a \in A'} s(a) = B$$

si y solo si $M' = \{m_i : a_i \in A'\}$ es un recubrimiento por trietas.

El Problema de la Partición

Datos: Un conjunto C y un tamaño para cada uno de sus elementos:

$$s : C \rightarrow \mathbb{N}$$

Pregunta: Determinar si existe un $C' \subseteq C$ tal que se verifique:

$$\sum_{a \in C'} s(a) = \sum_{a \in C \setminus C'} s(a)$$

Este es claramente un problema de NP: se eligen de forma no determinista los elementos de C' y en tiempo polinómico se determina si los tamaños totales de los conjuntos C' y $C \setminus C'$ son iguales.

Para demostrar que es NP-completo, vamos a reducir el problema SUMA a este problema.

Reducción SUMA a PARTICIÓN

Si tenemos un problema de suma con A , tamaños s y entero B , entonces creamos un problema de la partición en el que $C = A \cup \{b_1, b_2\}$ (añadimos dos valores nuevos).

Los tamaños en el problema de la PARTICIÓN de los elementos de A son los mismos que en el problema de la SUMA.

Los tamaños de los nuevos elementos, b_1 y b_2 , son:

$$\begin{aligned}s(b_1) &= \left(\sum_{i=1}^k s(a_i)\right) \\ s(b_2) &= 2B\end{aligned}$$

Cada uno de estos pesos necesita, a lo más, $(3pq)$ bits. Se pueden calcular zona a zona de forma consecutiva.

Conjunto C

El conjunto C es igual a $\{a_1, \dots, a_k, b_1, b_2\}$.

La suma de los pesos de sus elementos es

$$\begin{aligned}\sum_{x \in A} s(x) &= \sum_{i=1}^k s(a_i) + s(b_1) + s(b_2) = \\ \sum_{i=1}^k s(a_i) + \sum_{i=1}^k s(a_i) + 2B &= 2 \sum_{i=1}^k s(a_i) + 2B\end{aligned}$$

Este conjunto se parte en dos mitades cuando cada una pesa:

$$\sum_{i=1}^k s(a_i) + B$$

Si la solución es positiva al problema de la PARTICIÓN, entonces es positiva para la SUMA

Supongamos $C' \subseteq C$, tal que

$$\sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a)$$

Entonces cada una de estas sumas es $\sum_{i=1}^k s(a_i) + B$. Uno de los conjuntos (supongamos que es A') debe de contener b_1 y no b_2 .

La suma de los pesos de los elementos de C' distintos de b_1 tiene que ser B .

Por tanto, existe una solución positiva al problema de la SUMA.

Si la solución es positiva a SUMA, entonces es positiva al problema de la PARTICIÓN

Si el problema de la suma tiene solución con A' tal que $\sum_{a \in A'} = B$, entonces $C' = \{b_1\} \cup A'$ ya que la suma de los pesos de C' es

$$\left(\sum_{i=1}^k s(a_i) + B \right)$$

que es la mitad del total.